

EXAMGUARD

Secure Online Assessment, Examination & Live Invigilation Platform

FULL PRODUCT • UX • FUNCTIONAL • TECHNICAL • INFRASTRUCTURE DEVELOPER
HANDOFF

Audience: Product team • UX/UI designers • frontend developers • desktop-app developers • backend developers • DevOps • security • QA • infrastructure

Purpose

This specification defines the intended product behaviour so that implementation teams can build the platform without relying on assumptions. It covers registration, organisation onboarding, university/school identity, candidate approval, examination delivery, secure kiosk operation, live invigilation, recording, offline operation, synchronisation, reporting, backend architecture, databases, infrastructure and failure recovery.

Core principle

The platform must continue to protect and preserve an assessment when individual services fail. A temporary internet outage, authentication-service interruption or invigilator disconnection must not unnecessarily destroy an active examination attempt.

Version 2.0 • September 2026

1. Product Definition

ExamGuard is a multi-tenant assessment platform for universities, schools, employers, recruitment agencies, training providers, certification bodies and other organisations that need people to complete controlled online assessments.

The system consists of four major products:

Product	Primary responsibility
Admin / Organisation Portal	Organisation setup, people, exams, question banks, assignments, policies, reports, billing/integrations.
Secure Candidate Application	Locked-down exam environment, timer, questions, local storage, recording, offline operation, submission.
Invigilator Console	Monitor up to 10 candidates, live screen/camera/audio where permitted, voice contact, events and notes.
Platform Backend	Identity, exam delivery, synchronisation, recording metadata, audit, reporting, notifications, integrations.

Supported organisation modes

- University / college: students normally authenticate using institutional email/identity.
- School: learners may use school-managed accounts or invitation codes.
- Employer: applicants/employees may be invited by verified email or enterprise identity provider.
- Recruitment agency: agency manages candidates and can optionally give client organisations controlled access.
- Certification/training: candidates can be invited for a specific assessment session.
- Multi-tenant SaaS: every organisation's data must be logically isolated.

2. User Types & Permissions

Role	Can do
Platform Super Admin	Manage platform tenants, infrastructure-level configuration, support and operational controls.
Organisation Owner	Manage organisation, security policy, users, exams, integrations, reports and billing settings.
Organisation Admin	Manage exams, candidates, invigilators, assignments and reports according to permissions.
Exam Manager	Create exams, question banks, sessions, candidate assignments and results.
Invigilator	Monitor assigned candidates; max 10 active candidates; communicate and log events.
Reviewer / Marker	Review scripts, recordings, incidents and results without necessarily accessing live sessions.
Candidate / Student / Applicant	Authenticate, complete assigned assessment and submit.
Support Agent	Limited support access with explicit audited permissions; no unrestricted candidate data access.

Role-based access control

Use RBAC with optional organisation-level permission scopes. Do not hard-code permissions into individual screens. Backend APIs must enforce permissions independently of the UI.

Example permission keys

```
exam:create exam:publish candidate:approve invigilator:create live:view live:voice recording:view recording:download  
report:view result:release organisation:manage_users organisation:manage_security audit:view
```

3. Registration, Login & Identity

Authentication is one of the highest-risk areas for exam-day failure. The product should support multiple identity methods while giving organisations control over which methods are allowed.

Candidate onboarding options

Method	Use case
Institutional email	University/school students; candidate verifies their organisation email.
Organisation invitation	Candidate receives a single-use invitation link/code.
SSO	Universities, employers and larger organisations using SAML/OIDC identity providers.
Enterprise directory	Optional Microsoft/Google or other managed identity integrations where supported.
Exam access code	Controlled fallback for approved candidates; should not replace identity verification when stronger identity is required.

Candidate registration flow

1. Organisation selects candidate → 2. invitation issued → 3. candidate creates/links account 4. email/domain/identity verification → 5. candidate profile created → 6. organisation approval 7. assessment assigned → 8. pre-exam device check → 9. exam-day login → 10. secure exam session

University example

A university can configure its approved domains, e.g. student university email addresses. A student who attempts to register using an unapproved personal email should be routed to the organisation's configured verification/approval process rather than automatically becoming an approved candidate.

Login design

LOGIN Institution / Organisation: [Select] Email / Student ID: [_____] Password / SSO: [_____] [Continue] Alternative: [Sign in with institutional SSO] Need help? Exam access status • System status • Support

Avoid exam-day login failures

- Allow candidates to complete identity verification and device checks days before the assessment.
- Pre-create candidate exam entitlements rather than creating accounts at the start time.
- Support SSO where the organisation already operates an identity provider.
- Use short-lived exam-session tokens only after the candidate has authenticated.
- Cache the signed exam entitlement locally after a successful pre-check, subject to organisation policy.
- Provide a status page and operational dashboard before a major exam.
- Use multiple backend instances and managed database failover.
- Never depend on a single authentication server or single application server.

4. Candidate Approval & Exam Entitlement

The system must separate account existence, organisation approval and exam entitlement. A registered person is not automatically permitted to sit every assessment.

State	Meaning
Invited	Organisation has created an invitation; candidate has not completed onboarding.
Registered	Account exists but organisation verification may be incomplete.
Pending approval	Candidate requires organisation review.
Approved	Candidate is authorised by the organisation.
Exam assigned	Candidate has a specific exam entitlement/session.
Pre-check complete	Candidate has passed required readiness checks.
Exam active	Secure attempt is running.
Submitted	Candidate has submitted; evidence is being finalised/synchronised.
Completed	Server has received and verified the submission package.
Blocked / Suspended	Access is denied by organisation policy.

Candidate approval screen

CANDIDATE APPROVAL Sarah M. Student ID: 20261234 Institution email: sarah@university.example ✓ Verified Programme: BSc Computer Science Identity status: Verified Exam assigned: Mathematics 101 – 14 Oct 2026 09:00 [APPROVE] [REQUEST VERIFICATION] [REJECT] [VIEW AUDIT]

Pre-exam readiness

- Identity verified.
- Exam entitlement signed and valid.
- Device/application version accepted.
- Camera and microphone test passed if required.
- Screen capture test passed if required.
- Storage capacity check passed.
- Clock/time synchronisation completed.
- Connectivity test completed or offline policy explicitly enabled.
- Candidate sees a clear support route before exam day.

5. Organisation Dashboard & Navigation

EXAMGUARD Dashboard EXAMS All Exams • Create Exam • Question Bank • Sessions PEOPLE Candidates • Groups • Invigilators • Reviewers LIVE Invigilation • Active Sessions RESULTS Results • Attempts • Scripts • Recordings REPORTS Security Events • Blackouts • Performance • Audit ORGANISATION Users • Roles • Security • Branding • Integrations • Billing

Dashboard wireframe

[Organisation](#) ▾ [Admin](#) ■ [CANDIDATES](#) [LIVE](#) [INCIDENTS](#) ■ ■ [Exams](#) ■ 12 438 27 3 ■ ■ [Candidates](#) ■ ■ ■ [Invigilators](#) ■
[UPCOMING EXAMS](#) ■ ■ [Live](#) ■ [Mathematics](#) 101 120 candidates 09:00 ■ ■ [Results](#) ■ [Recruitment Test](#) 84 candidates 13:00 ■ ■
[Reports](#) ■ ■ ■ [Settings](#) ■ [SYSTEM HEALTH: API](#) ✓ [DB](#) ✓ [STORAGE](#) ✓ [MEDIA](#) ✓ ■

Admin functions

- Create/edit/archive assessments.
- Import candidates via CSV/API/SSO group.
- Add and approve invigilators.
- Set maximum invigilator capacity: 10.
- Create examination sessions.
- Assign or automatically distribute candidates.
- View system health and active incidents.
- Export reports.

6. Exam Creation & Question Bank

Exam creation should be a guided workflow with validation before publication.

CREATE EXAM ① Details → ② Questions → ③ Timing → ④ Security → ⑤ Invigilation → ⑥ Review → ⑦ Publish	
Configuration	Examples
Exam identity	Name, code, description, organisation, subject/programme.
Questions	MCQ, multiple response, true/false, short answer, essay, file upload where supported.
Question bank	Pools, randomisation, categories, difficulty, versions.
Timing	Duration, start window, late-entry policy, breaks, automatic submission.
Security	Kiosk, screen capture, camera, microphone, event monitoring.
Invigilation	Required/not required, max candidates per invigilator, rotation interval, communication policy.
Offline	Allowed/not allowed, maximum offline duration, sync policy.
Results	Auto-marking, manual marking, release date, moderation.

Exam publish gate

- Every published exam receives an immutable version identifier.
- Changes after publishing create a new version rather than silently changing an active exam.
- Server signs the exam package/manifest.
- Candidate devices verify package integrity before starting.
- Questions and answers are not trusted from the client; the server validates submission structure.

7. Invigilator Management & Allocation

Administrators can add invigilators individually or by import and assign them to one or more examination sessions.

ADD INVIGILATOR Name: [_____] Email / Staff ID: [_____] Role: [Invigilator ▼] Status: [Active ▼] Permissions: [Live] [Voice] [Recording Review] [Reports] [INVITE INVIGILATOR]

Capacity rule

Hard limit: one invigilator may have a maximum of 10 simultaneously active/assigned candidates in the live console. The backend must enforce this limit, not merely the UI.

Allocation algorithm

- Find available active invigilators for the session.
- Calculate current active assignment count.
- Allocate each candidate to an invigilator with fewer than 10 candidates.
- Use configurable randomisation/rotation when random monitoring is enabled.
- Record the allocation in the audit log.
- If all invigilators are at capacity, place the candidate in an unassigned queue and alert the session administrator.
- If an invigilator disconnects, preserve assignments and reassign only according to configured failover policy.

Invigilator status

Status	Meaning
Available	Can receive candidates.
Monitoring	Has 1-10 active candidates.
At capacity	Has exactly 10.
Disconnected	Live connection unavailable; failover may be triggered.
Paused	Temporarily not receiving assignments.
Suspended	Cannot access invigilation.

8. Live Invigilation Console

LIVE SESSION 37

ACTIVE MY LOAD 8/10 3 FLAGGED ■■■■

CANDIDATES ■ LIVE

SCREEN ■ CANDIDATE ■ Sarah M. ■ Sarah ■ [SCREEN PREVIEW] ■ John ■ [CAMERA] ■ David ■ Mic ● ON ■ Maria ■ Network ● GOOD ■ Thabo ■ Timer 01:14:23 ■ ... ■■■■

EVENTS: blackout • camera interruption • focus loss ■ [TALK] [MESSAGE] [FLAG] [SNAPSHOT] [TIMELINE] ■■■■

Live functions

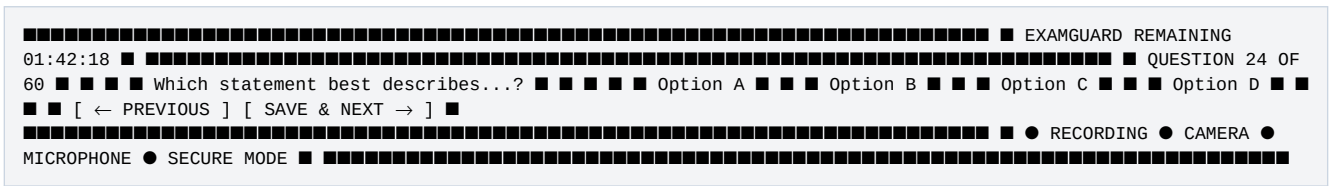
- See candidate list and status at a glance.
- Select any assigned candidate.
- View live screen where permitted.
- View camera and audio status.
- Start/stop two-way voice communication.
- Send a text message.
- Flag an event and add a note.
- View the candidate's live event timeline.
- See connection health.
- Never expose candidates outside the invigilator's authorised assignment scope.

Voice interaction

The voice connection should be explicitly initiated by the invigilator. The candidate receives a visible/audio indication according to the configured policy. The start/end timestamps and invigilator identity are recorded. Voice communications should follow the organisation's recording/retention policy.

9. Candidate Secure Application

For strong lockdown, use a dedicated desktop application with a web-based UI layer and native OS integration rather than relying on a normal browser tab.



Candidate functions

- Secure startup and exam entitlement validation.
- Device readiness checks.
- Countdown timer.
- Question navigation according to policy.
- Autosave every meaningful answer/state change.
- Local encrypted state.
- Screen/camera/microphone recording where configured.
- Connection indicator.
- Offline continuation.
- Automatic submission at timer expiry if configured.
- Submission receipt.
- Recovery after application restart where policy permits.

10. Pre-Exam Device Check

DEVICE CHECK ✓ Account verified ✓ Exam entitlement verified ✓ Application version accepted ✓ Camera detected ✓ Microphone detected ✓ Screen capture ready ✓ Storage available ✓ System clock synchronised ✓ Network test complete [RUN FULL TEST] [CONTACT SUPPORT]

Preflight should be available before exam day

The candidate should be able to run the full check well before the scheduled assessment. Results are stored against the candidate/device/session readiness record. If a test fails, the organisation can see the failure before the assessment starts.

Device requirements should be configurable

- Minimum supported operating system versions.
- Minimum RAM/CPU guidance.
- Minimum free storage for local recordings.
- Supported camera/microphone devices.
- Supported network conditions.
- Required application version.
- Whether external monitors are allowed.
- Whether virtual machines are prohibited by policy.
- Whether offline mode is permitted.

11. Offline-First Examination Engine

The exam must not depend on a continuous internet connection. The local application should maintain an authoritative local session state while the server maintains the authoritative submitted record after synchronisation.

LOCAL SESSION Exam manifest + signed entitlement ↓ Encrypted local database ↓ Answers + timer state + events + recording chunks ↓ Sync queue ↓ Resumable upload when network returns

Offline rules

- Answers are persisted locally immediately.
- Recording is chunked rather than kept as one huge file.
- Each chunk has an identifier, sequence number and integrity checksum.
- Upload is resumable and idempotent.
- Duplicate uploads must not create duplicate evidence.
- Clock drift is detected and logged.
- Server time is used whenever reachable; local monotonic time protects timer continuity during outages.
- An organisation can set a maximum permitted offline duration.
- Submission is queued until required evidence is confirmed uploaded or policy permits deferred upload.

12. Recording & Evidence Pipeline

CAMERA ████████ MIC ████████████████████ RECORDING / ENCODING ███ CHUNKS ███ LOCAL ENCRYPTED STORE SCREEN ██████████
█ ▼ UPLOAD QUEUE █ ▼ OBJECT STORAGE █ ▼ EVIDENCE INDEX / DB

Do not upload one giant file at the end

Recordings should be chunked and uploaded in the background. This allows recovery from network failures, application interruptions and server-side upload errors.

Evidence metadata

Field	Purpose
attempt_id	Links evidence to one examination attempt.
stream_type	screen / camera / audio.
sequence	Ordering of chunks.
start/end time	Timeline alignment.
checksum	Integrity verification.
storage key	Object-storage location.
upload state	Pending / uploading / uploaded / failed.
retention state	Retention/deletion lifecycle.

13. Recommended Backend & Infrastructure

The platform should be deployed as a highly available service rather than a single server.



Recommended components

Layer	Recommendation
Frontend web	React + TypeScript or equivalent.
Desktop candidate app	Tauri/Electron-style architecture with native OS integration; final choice should follow target OS and lockdown requirements.
Backend API	TypeScript/Node.js, Go, Java, .NET or equivalent; use a stateless service design.
Primary database	PostgreSQL.
Cache / queues	Redis or managed equivalent.
Object storage	S3-compatible durable object storage.
Live communication	WebRTC with signalling service and TURN servers.
Search/analytics	Start with PostgreSQL; add OpenSearch/warehouse only when scale requires it.
CDN/WAF	Managed CDN and web application firewall.
Observability	Central logs, metrics, traces, uptime monitoring and alerting.

14. What Servers/Services Are Needed?

For a production deployment, the following logical services should exist. They do not necessarily need to be separate physical machines; managed cloud services are preferred where practical.

Service	Purpose	Availability requirement
Web frontend hosting	Admin/invigilator portal	CDN/high availability
API service	Core application APIs	Multiple instances
Authentication / identity	Login, SSO, MFA, tokens	Highly available; avoid single-instance dependency
PostgreSQL	Users, exams, attempts, audit metadata	Managed HA + backups
Redis	Cache, rate limits, transient session/job state	Managed HA or equivalent
Queue/worker service	Email, sync, report generation, media metadata	Retryable jobs
Object storage	Recordings and evidence	Durable + lifecycle policies
TURN servers	WebRTC connectivity fallback	Multiple regional/zone endpoints
WebRTC signalling	Negotiates live media sessions	Multiple instances
Notification service	Email and operational alerts	Provider redundancy where required
Monitoring/logging	Health, errors, security events	Centralised + retained
Backup service	Database and configuration recovery	Automated + tested restores

Do not build exam-day infrastructure as a single VPS

A single server creates a single point of failure. Use managed PostgreSQL, multiple stateless API instances, durable object storage, redundant authentication and monitoring, and tested backups. For large examination events, capacity should be load-tested using realistic concurrent candidate and recording workloads.

15. Core Database Model

organisations users roles permissions organisation_users candidates invigilators exams exam_versions questions question_options exam_questions sessions exam_assignments attempts answers recording_streams recording_chunks events invigilation_assignments invigilation_contacts submissions results notifications audit_logs support_cases integration_configs

Important relationships

Entity	Key relationships
Organisation	owns users, exams, candidates, sessions, policies.
User	may belong to one or more organisations through organisation_users.
Candidate	links to a user identity and organisation profile.
Exam	has immutable versions.
Session	is a scheduled run of an exam/version.
Assignment	connects candidate to session.
Attempt	records the candidate's actual run.
Answer	belongs to attempt/question.
Recording chunk	belongs to attempt/stream.
Event	belongs to attempt and may reference an invigilator.
Invigilation assignment	connects invigilator to candidate/session and enforces max 10.
Submission	represents final submission package and verification state.
Audit log	immutable operational/security trail.

16. Backend API / Service Boundaries

```
AUTH POST /auth/login POST /auth/refresh POST /auth/sso/callback ORGANISATIONS GET /organisations/:id POST
/organisations/:id/users EXAMS POST /exams POST /exams/:id/publish GET /exams/:id CANDIDATES POST /candidates/import
POST /candidates/:id/approve POST /assignments SESSIONS POST /sessions GET /sessions/:id/status ATTEMPTS POST
/attempts/start PATCH /attempts/:id/state POST /attempts/:id/submit RECORDINGS POST /attempts/:id/chunks POST
/attempts/:id/chunks/complete INVIGILATION GET /live/sessions/:id POST /live/assignments POST /live/:candidateId/voice
POST /live/:candidateId/events REPORTS GET /reports/incidents GET /reports/blackouts GET /attempts/:id/timeline
```

API requirements

- Authentication and authorisation on every protected endpoint.
- Tenant isolation enforced server-side.
- Idempotency keys for attempt state and recording uploads.
- Rate limiting and abuse protection.
- Request validation using explicit schemas.
- Audit logging for privileged operations.
- Pagination for all list endpoints.
- Versioned APIs for long-lived integrations.

17. Failure & Recovery Design

Failure	Expected behaviour
Candidate internet loss	Exam continues offline; event logged; uploads queue.
Candidate application restart	Recover local session if policy permits; restore last saved state.
Candidate device crash	Recover from local persisted state where possible; log interruption.
Camera failure	Continue according to organisation policy; flag event; attempt reconnection.
Microphone failure	Continue according to policy; flag event; attempt reconnection.
Screen capture failure	Flag high-priority event; attempt recovery; preserve exam state.
Invigilator disconnects	Candidates remain active; system can reassign according to policy.
API instance failure	Load balancer routes to another instance.
Database primary failure	Managed failover/replica strategy restores service.
Object-storage upload failure	Chunk remains queued and retries.
Auth provider outage	Do not invent unsafe bypasses; use pre-approved exam entitlement/session recovery policy.
Full platform outage	Candidate local session should preserve state; operational recovery procedures determine continuation/submission policy.

Exam-day operational rule

The platform must have a documented incident mode. Administrators should be able to see whether the problem is local to a candidate, regional, service-wide, authentication-related, storage-related or live-media-related.

18. Reports & Analytics

Reports must be factual and traceable. Technical events should be presented as events rather than automatically treated as misconduct findings.

Report	Contents
Candidate attempt report	Identity, exam, start/end, answers, score, submission status, evidence status.
Blackout report	Internet interruptions, duration, count, recovery, sync state.
Recording health	Camera/mic/screen availability, missing chunks, upload status.
Invigilation report	Invigilator load, assignments, contacts, notes, flagged events.
Session health	Active candidates, service health, unresolved incidents.
Organisation report	Completion, results, candidate status and technical event summaries.
Audit report	Who changed what, when, and from which authenticated account/session.

Candidate timeline wireframe

CANDIDATE: Sarah M. ATTEMPT: A-10482 09:00:00 Exam started 09:41:22 Internet disconnected – 2m45s 09:44:07 Internet restored 09:44:08 Sync resumed 10:02:18 Invigilator voice contact – 00:31 10:14:03 Camera interruption – 00:08 10:47:13 Submitted 10:48:02 Evidence verified

19. Security, Privacy & Governance

- TLS for all network communication.
- Encryption at rest for databases and object storage.
- Strong tenant isolation.
- RBAC and least-privilege access.
- MFA for administrators and invigilators where required.
- Short-lived access tokens and secure refresh-token handling.
- Secrets stored in a managed secret store rather than source code.
- Audit logs for privileged actions.
- Recording access should itself be audited.
- Configurable retention and deletion policies.
- Data export/deletion workflows subject to organisation policy and applicable law.
- Explicit candidate notice/consent language where required.
- Separate production, staging and development environments.
- Regular dependency, vulnerability and penetration testing.
- Backups must be encrypted and restore-tested.

Important product boundary

No technical system should claim that a normal application can defeat operating-system-level controls such as a forced shutdown or security screen. Strong lockdown should combine a dedicated application with managed-device/kiosk capabilities where the organisation requires them.

20. Notifications & Communication

Notification	Recipients
Invitation	Candidate
Approval/rejection	Candidate / administrator
Pre-exam reminder	Candidate
Readiness failure	Candidate + support/admin according to policy
Exam start reminder	Candidate / invigilator
Invigilator capacity alert	Session administrator
Candidate blackout alert	Invigilator / administrator depending on severity
Submission received	Candidate / administrator
Submission evidence incomplete	Administrator / operations
Service incident	Operations / administrators

Communication channels

Use in-app notifications for immediate events, email for scheduled/administrative communication, and optional SMS/push only where an organisation requires it. Critical exam controls should never depend exclusively on email delivery.

21. Deployment, DevOps & Operations

Recommended production topology

DNS → CDN/WAF → Load Balancer → API instances ■→ PostgreSQL HA ■→ Redis ■→ Job Queue / Workers ■→ Object Storage
■→ Notification Provider ■→ WebRTC Signalling → TURN cluster

Required operational controls

- CI/CD with automated tests and security checks.
- Blue/green or rolling deployments where appropriate.
- Database migration tooling with rollback strategy.
- Environment configuration separated from code.
- Centralised structured logging.
- Metrics for API latency, error rate, DB connections, queue depth, storage uploads and WebRTC sessions.
- Distributed tracing for critical flows.
- Uptime checks for public and internal services.
- Alerts with escalation paths before major exam sessions.
- Capacity/load tests using concurrent candidates and realistic recording traffic.
- Pre-event infrastructure readiness checklist.

Exam-event checklist

24-72 HOURS BEFORE ✓ candidate roster finalised ✓ invigilators assigned ✓ exam version locked ✓ device checks available ✓ authentication/SSO tested ✓ storage capacity checked ✓ TURN/media capacity checked ✓ database backups verified ✓ monitoring alerts tested ✓ support contacts confirmed ON EXAM DAY ✓ service health green ✓ candidate readiness monitored ✓ invigilator load ≤10 ✓ incidents triaged ✓ submission/evidence queue monitored

22. QA & Test Matrix

Area	Must test
Authentication	Correct login, wrong credentials, expired invitation, SSO failure, MFA, logout.
Approval	Pending/approved/rejected/suspended transitions.
Exam delivery	Timing, navigation, answer persistence, autosave, auto-submit.
Offline	Disconnect before start, during question, during upload, long outage, reconnect.
Recording	Camera/mic/screen start, interruption, recovery, chunk ordering, missing chunk.
Invigilation	10-candidate limit, allocation, rotation, reassignment, voice contact.
Security	Tenant isolation, RBAC, token expiry, privilege escalation attempts.
Database	Failover, backup, restore, migration.
Load	Concurrent candidates, recording uploads, live viewers, peak login.
Recovery	Application restart, API restart, DB failover, object-storage retry.
Accessibility	Keyboard support where compatible, readable status, captions/text alternatives where required.

Critical acceptance tests

- A candidate cannot accidentally lose saved answers because the internet disappears.
- An invigilator cannot be assigned an 11th active candidate.
- A candidate from Organisation A cannot access Organisation B data.
- Duplicate recording chunks do not corrupt the final evidence timeline.
- Server restart does not invalidate active sessions unnecessarily.
- Exam version cannot change silently after candidates begin.
- All privileged actions are auditable.
- Candidate can see the recording/security status required by the organisation's policy.

23. Accessibility & Candidate Support

- Support accessible question content and keyboard navigation where assessment policy allows.
- Do not make accessibility accommodations indistinguishable from security exceptions; configure them explicitly.
- Provide clear error messages and recovery instructions.
- Provide a pre-exam support route.
- Allow organisation-defined accommodations such as additional time where permitted.
- Ensure timer, recording and security indicators have non-colour-only status cues.
- Provide text communication as an alternative to voice contact where required.

Support model

Candidate → Organisation Support → Platform Support The product should distinguish organisation-specific issues (exam rules, eligibility, accommodation) from platform issues (login outage, application failure, service incident).

24. Development Roadmap

Phase	Deliverable
MVP-1	Multi-tenant organisation accounts, users, roles, candidate registration/approval.
MVP-2	Exam creation, question bank, assignment and candidate secure app.
MVP-3	Timer, autosave, local encrypted state and submission.
MVP-4	Camera/mic/screen recording and chunked evidence pipeline.
MVP-5	Offline-first sync and recovery.
MVP-6	Invigilator management, max 10 allocation and live console.
MVP-7	Voice contact, events, blackout reports and audit.
MVP-8	Results, recording review, exports, integrations and production hardening.

Future extensions

- University SIS/LMS integration.
- Microsoft/Google/SAML/OIDC SSO.
- HR/recruitment ATS integrations.
- Automated candidate imports.
- Advanced analytics.
- Question randomisation and adaptive testing.
- Human-reviewed integrity workflows.
- Mobile-specific assessment modes where appropriate.
- Regional deployment/data residency options.
- Organisation API and webhooks.

25. End-to-End System Blueprint

[illegible]

Developer definition of done

- Every major screen in this document has a corresponding UI specification and state model.
- Every user action is mapped to a backend permission.
- Every exam attempt has durable local state.
- Every recording stream has resumable chunk storage.
- Every important operational/security event is timestamped and auditable.
- Invigilator maximum of 10 is enforced server-side.
- Authentication and candidate entitlement are resilient enough for planned exam-day operation.
- Production infrastructure has redundancy, backups, monitoring and tested recovery.
- Security/privacy requirements are documented before production launch.
- Load testing demonstrates capacity for the organisation's planned peak examination population.

Final implementation note

This document is the product blueprint, not a substitute for detailed platform-specific security review, legal/privacy review, accessibility review or OS-kiosk validation. The development team should convert each section into tickets, API contracts, database migrations, UI states, automated tests and operational runbooks before production launch.